

GrocerIQ Complete Implementation Guide

Phase 1: Project Setup & Backend Foundation

Step 1: Initialize Project Structure

```
bash

# Create main project directory
mkdir groceriq
cd groceriq

# Initialize backend
mkdir backend
cd backend
npm init -y

# Install backend dependencies
npm install express mongoose cors dotenv helmet morgan
npm install @clerk/clerk-sdk-node
npm install axios
npm install -D nodemon concurrently

# Create frontend
cd ..
npx create-react-app frontend
cd frontend

# Install frontend dependencies
npm install @mui/material @emotion/react @emotion/styled
npm install @mui/icons-material
npm install @clerk/clerk-react
npm install axios react-router-dom
npm install recharts

# Go back to root
cd ..
```

Step 2: Backend Setup

Environment Configuration (.env)

env

MongoDB

MONGODB_URI=mongodb+srv://your-username:password@cluster.mongodb.net/groceriq

Clerk

CLERK_SECRET_KEY=sk_test_your_clerk_secret_key

CLERK_PUBLISHABLE_KEY=pk_test_your_clerk_publishable_key

Hugging Face

HUGGING_FACE_API_KEY=hf_your_hugging_face_token

Server

PORT=5000

NODE_ENV=development

Backend Package.json Scripts

json

```
{
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js",
    "client": "cd ../frontend && npm start",
    "dev:full": "concurrently \"npm run dev\" \"npm run client\""
  }
}
```

Step 3: Database Models

models/User.js

javascript

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
  clerkId: {
    type: String,
    required: true,
    unique: true
  },
  email: {
    type: String,
    required: true
  },
  firstName: String,
  lastName: String,
  onboardingComplete: {
    type: Boolean,
    default: false
  },
  createdAt: {
    type: Date,
    default: Date.now
  }
});

module.exports = mongoose.model('User', userSchema);
```

models/UserPreferences.js

javascript

```
const mongoose = require('mongoose');

const userPreferencesSchema = new mongoose.Schema({
  userId: {
    type: String,
    required: true,
    unique: true
  },
  dietaryRestrictions: [{
    type: String,
    enum: ['vegetarian', 'vegan', 'gluten-free', 'dairy-free', 'nut-free', 'keto', 'paleo']
  }],
  cuisinePreferences: [String],
  cookingFrequency: {
    type: String,
    enum: ['daily', 'few-times-week', 'weekly', 'rarely'],
    default: 'few-times-week'
  },
  householdSize: {
    type: Number,
    default: 1,
    min: 1,
    max: 10
  },
  shoppingFrequency: {
    type: String,
    enum: ['daily', 'weekly', 'bi-weekly', 'monthly'],
    default: 'weekly'
  },
  allergies: [String],
  preferredStores: [String],
  updatedAt: {
    type: Date,
    default: Date.now
  }
});

module.exports = mongoose.model('UserPreferences', userPreferencesSchema);
```

models/Budget.js

javascript

```
const mongoose = require('mongoose');

const budgetSchema = new mongoose.Schema({
  userId: {
    type: String,
    required: true,
    unique: true
  },
  amount: {
    type: Number,
    required: true,
    min: 0
  },
  duration: {
    type: String,
    enum: ['weekly', 'bi-weekly', 'monthly'],
    default: 'weekly'
  },
  priority: {
    type: String,
    enum: ['strict', 'flexible', 'loose'],
    default: 'flexible'
  },
  spent: {
    type: Number,
    default: 0
  },
  lastReset: {
    type: Date,
    default: Date.now
  },
  updatedAt: {
    type: Date,
    default: Date.now
  }
});

module.exports = mongoose.model('Budget', budgetSchema);
```

models/GroceryItem.js


```
const mongoose = require('mongoose');

const groceryItemSchema = new mongoose.Schema({
  userId: {
    type: String,
    required: true
  },
  name: {
    type: String,
    required: true
  },
  category: {
    type: String,
    required: true,
    enum: ['produce', 'dairy', 'meat', 'pantry', 'frozen', 'beverages', 'snacks', 'other']
  },
  quantity: {
    type: Number,
    default: 1
  },
  unit: {
    type: String,
    default: 'piece'
  },
  estimatedPrice: {
    type: Number,
    default: 0
  },
  expiryDate: Date,
  isPurchased: {
    type: Boolean,
    default: false
  },
  isRecurring: {
    type: Boolean,
    default: false
  },
  recurringInterval: {
    type: String,
    enum: ['weekly', 'bi-weekly', 'monthly'],
    default: 'weekly'
  },
  priority: {
    type: String,
    enum: ['low', 'medium', 'high'],
    default: 'medium'
  }
});
```

```
    },
    notes: String,
    createdAt: {
      type: Date,
      default: Date.now
    },
    updatedAt: {
      type: Date,
      default: Date.now
    }
  }
});

module.exports = mongoose.model('GroceryItem', groceryItemSchema);
```

Step 4: Express Server Setup

server.js


```

const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
const helmet = require('helmet');
const morgan = require('morgan');
require('dotenv').config();

const app = express();

// Middleware
app.use(helmet());
app.use(cors({
  origin: process.env.NODE_ENV === 'production'
    ? 'https://your-frontend-domain.vercel.app'
    : 'http://localhost:3000',
  credentials: true
}));
app.use(morgan('combined'));
app.use(express.json({ limit: '10mb' }));

// MongoDB Connection
mongoose.connect(process.env.MONGODB_URI, {
  useNewUrlParser: true,
  useUnifiedTopology: true,
})
.then(() => console.log('MongoDB connected'))
.catch(err => console.error('MongoDB connection error:', err));

// Routes
app.use('/api/auth', require('./routes/auth'));
app.use('/api/users', require('./routes/users'));
app.use('/api/preferences', require('./routes/preferences'));
app.use('/api/budget', require('./routes/budget'));
app.use('/api/grocery', require('./routes/grocery'));
app.use('/api/ai', require('./routes/ai'));

// Health check
app.get('/api/health', (req, res) => {
  res.json({ status: 'OK', timestamp: new Date().toISOString() });
});

// Error handling middleware
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({
    message: 'Something went wrong!',
  });
});

```

```
      error: process.env.NODE_ENV === 'development' ? err.message : {}
    });
  });

  const PORT = process.env.PORT || 5000;
  app.listen(PORT, () => {
    console.log(`Server running on port ${PORT}`);
  });
```

middleware/auth.js

javascript

```
const { ClerkExpressRequireAuth } = require('@clerk/clerk-sdk-node');

const requireAuth = ClerkExpressRequireAuth({
  onError: (error) => {
    console.error('Auth error:', error);
    return {
      status: 401,
      message: 'Unauthorized'
    };
  }
});

module.exports = requireAuth;
```

Step 5: API Routes

routes/auth.js

javascript

```
const express = require('express');
const { Webhook } = require('svix');
const User = require('../models/User');

const router = express.Router();

// Clerk webhook for user creation/updates
router.post('/webhook', express.raw({ type: 'application/json' })), async (req, res) => {
  try {
    const WEBHOOK_SECRET = process.env.CLERK_WEBHOOK_SECRET;

    if (!WEBHOOK_SECRET) {
      throw new Error('Please add CLERK_WEBHOOK_SECRET to .env');
    }

    const headers = req.headers;
    const payload = req.body;

    const wh = new Webhook(WEBHOOK_SECRET);
    const evt = wh.verify(payload, headers);

    const { id, email_addresses, first_name, last_name } = evt.data;

    if (evt.type === 'user.created') {
      await User.create({
        clerkId: id,
        email: email_addresses[0].email_address,
        firstName: first_name,
        lastName: last_name
      });
    }

    res.status(200).json({ message: 'Webhook processed' });
  } catch (error) {
    console.error('Webhook error:', error);
    res.status(400).json({ error: 'Webhook failed' });
  }
});

module.exports = router;
```

routes/users.js

javascript

```
const express = require('express');
const requireAuth = require('../middleware/auth');
const User = require('../models/User');

const router = express.Router();

// Get current user
router.get('/me', requireAuth, async (req, res) => {
  try {
    const user = await User.findOne({ clerkId: req.auth.userId });
    if (!user) {
      return res.status(404).json({ message: 'User not found' });
    }
    res.json(user);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
});

// Update onboarding status
router.patch('/onboarding', requireAuth, async (req, res) => {
  try {
    const user = await User.findOneAndUpdate(
      { clerkId: req.auth.userId },
      { onboardingComplete: true },
      { new: true }
    );
    res.json(user);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
});

module.exports = router;
```

routes/preferences.js

javascript

```
const express = require('express');
const requireAuth = require('../middleware/auth');
const UserPreferences = require('../models/UserPreferences');

const router = express.Router();

// Get user preferences
router.get('/', requireAuth, async (req, res) => {
  try {
    const preferences = await UserPreferences.findOne({ userId: req.auth.userId });
    res.json(preferences || {});
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
});

// Create or update preferences
router.post('/', requireAuth, async (req, res) => {
  try {
    const preferences = await UserPreferences.findOneAndUpdate(
      { userId: req.auth.userId },
      { ...req.body, userId: req.auth.userId, updatedAt: new Date() },
      { upsert: true, new: true }
    );
    res.json(preferences);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
});

module.exports = router;
```

routes/budget.js

javascript

```
const express = require('express');
const requireAuth = require('../middleware/auth');
const Budget = require('../models/Budget');

const router = express.Router();

// Get user budget
router.get('/', requireAuth, async (req, res) => {
  try {
    const budget = await Budget.findOne({ userId: req.auth.userId });
    res.json(budget || {});
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
});

// Create or update budget
router.post('/', requireAuth, async (req, res) => {
  try {
    const budget = await Budget.findOneAndUpdate(
      { userId: req.auth.userId },
      { ...req.body, userId: req.auth.userId, updatedAt: new Date() },
      { upsert: true, new: true }
    );
    res.json(budget);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
});

module.exports = router;
```

routes/grocery.js


```

const express = require('express');
const requireAuth = require('../middleware/auth');
const GroceryItem = require('../models/GroceryItem');

const router = express.Router();

// Get all grocery items for user
router.get('/', requireAuth, async (req, res) => {
  try {
    const { category, purchased, priority } = req.query;
    const filter = { userId: req.auth.userId };

    if (category) filter.category = category;
    if (purchased !== undefined) filter.isPurchased = purchased === 'true';
    if (priority) filter.priority = priority;

    const items = await GroceryItem.find(filter).sort({ createdAt: -1 });
    res.json(items);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
});

// Create grocery item
router.post('/', requireAuth, async (req, res) => {
  try {
    const item = new GroceryItem({
      ...req.body,
      userId: req.auth.userId
    });
    await item.save();
    res.status(201).json(item);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
});

// Update grocery item
router.patch('/:id', requireAuth, async (req, res) => {
  try {
    const item = await GroceryItem.findOneAndUpdate(
      { _id: req.params.id, userId: req.auth.userId },
      { ...req.body, updatedAt: new Date() },
      { new: true }
    );
    if (!item) {

```

```
        return res.status(404).json({ message: 'Item not found' });
    }
    res.json(item);
} catch (error) {
    res.status(500).json({ message: error.message });
}
});
```

// Delete grocery item

```
router.delete('/:id', requireAuth, async (req, res) => {
    try {
        const item = await GroceryItem.findOneAndDelete({
            _id: req.params.id,
            userId: req.auth.userId
        });
        if (!item) {
            return res.status(404).json({ message: 'Item not found' });
        }
        res.json({ message: 'Item deleted' });
    } catch (error) {
        res.status(500).json({ message: error.message });
    }
});
```

```
module.exports = router;
```

routes/ai.js


```

const express = require('express');
const axios = require('axios');
const requireAuth = require('../middleware/auth');
const UserPreferences = require('../models/UserPreferences');
const Budget = require('../models/Budget');

const router = express.Router();

// Generate grocery list using AI
router.post('/generate-list', requireAuth, async (req, res) => {
  try {
    const [preferences, budget] = await Promise.all([
      UserPreferences.findOne({ userId: req.auth.userId }),
      Budget.findOne({ userId: req.auth.userId })
    ]);

    // Build prompt based on user data
    const prompt = buildGroceryPrompt(preferences, budget);

    // Call Hugging Face API
    const response = await axios.post(
      'https://api-inference.huggingface.co/models/microsoft/DialoGPT-medium',
      { inputs: prompt },
      {
        headers: {
          'Authorization': `Bearer ${process.env.HUGGING_FACE_API_KEY}`,
          'Content-Type': 'application/json'
        }
      }
    );

    // Parse and structure the response
    const groceryList = parseAIResponse(response.data);

    res.json({ groceryList, prompt: prompt.substring(0, 100) + '...' });
  } catch (error) {
    console.error('AI generation error:', error);
    res.status(500).json({
      message: 'Failed to generate grocery list',
      error: error.response?.data || error.message
    });
  }
});

function buildGroceryPrompt(preferences, budget) {
  let prompt = "Generate a grocery shopping list with the following requirements:\n";

```

```

if (budget) {
  prompt += `Budget: ${budget.amount} for ${budget.duration}\n`;
}

if (preferences) {
  if (preferences.householdSize) {
    prompt += `Household size: ${preferences.householdSize} people\n`;
  }

  if (preferences.dietaryRestrictions?.length) {
    prompt += `Dietary restrictions: ${preferences.dietaryRestrictions.join(', ')}\n`;
  }

  if (preferences.allergies?.length) {
    prompt += `Allergies: ${preferences.allergies.join(', ')}\n`;
  }

  if (preferences.cookingFrequency) {
    prompt += `Cooking frequency: ${preferences.cookingFrequency}\n`;
  }
}

prompt += "\nReturn a JSON array of grocery items with name, category, quantity, unit, and es

return prompt;
}

function parseAIResponse(aiResponse) {
  // This is a simplified parser - you'd want more robust parsing
  try {
    // Try to extract JSON from the response
    const text = Array.isArray(aiResponse) ? aiResponse[0]?.generated_text || '' : aiResponse;
    const jsonMatch = text.match(/[[\s\S]*\]/);

    if (jsonMatch) {
      return JSON.parse(jsonMatch[0]);
    }

    // Fallback: return a basic grocery list
    return [
      { name: "Bananas", category: "produce", quantity: 6, unit: "pieces", estimatedPrice: 3.99 },
      { name: "Milk", category: "dairy", quantity: 1, unit: "gallon", estimatedPrice: 4.29 },
      { name: "Bread", category: "pantry", quantity: 1, unit: "loaf", estimatedPrice: 2.99 },
      { name: "Chicken Breast", category: "meat", quantity: 2, unit: "lbs", estimatedPrice: 8.99 },
      { name: "Rice", category: "pantry", quantity: 1, unit: "bag", estimatedPrice: 3.49 }
    ];
  }
}

```

```
    } catch (error) {  
      console.error('Failed to parse AI response:', error);  
      return [];  
    }  
  }  
}  
  
module.exports = router;
```

Phase 2: Frontend Development

Step 1: Frontend Project Structure

```
frontend/  
├── src/  
│   ├── components/  
│   │   ├── layout/  
│   │   ├── grocery/  
│   │   ├── preferences/  
│   │   └── common/  
│   ├── pages/  
│   ├── hooks/  
│   ├── utils/  
│   └── contexts/
```

Step 2: Main App Setup

src/App.js


```

import React from 'react';
import { BrowserRouter as Router, Routes, Route } from 'react-router-dom';
import { ClerkProvider, SignedIn, SignedOut } from '@clerk/clerk-react';
import { ThemeProvider, createTheme } from '@mui/material/styles';
import CssBaseline from '@mui/material/CssBaseline';

import Layout from './components/layout/Layout';
import LandingPage from './pages/LandingPage';
import Dashboard from './pages/Dashboard';
import Onboarding from './pages/Onboarding';
import Preferences from './pages/Preferences';
import Budget from './pages/Budget';
import Analytics from './pages/Analytics';
import RequireOnboarding from './components/RequireOnboarding';

const theme = createTheme({
  palette: {
    mode: 'light',
    primary: {
      main: '#2e7d32',
    },
    secondary: {
      main: '#ff9800',
    },
  },
});

const clerkPubKey = process.env.REACT_APP_CLERK_PUBLISHABLE_KEY;

function App() {
  return (
    <ClerkProvider publishableKey={clerkPubKey}>
      <ThemeProvider theme={theme}>
        <CssBaseline />
        <Router>
          <div className="App">
            <SignedOut>
              <LandingPage />
            </SignedOut>
            <SignedIn>
              <Layout>
                <Routes>
                  <Route path="/" element={
                    <RequireOnboarding>
                      <Dashboard />
                    </RequireOnboarding>
                  />
                </Routes>
              </Layout>
            </SignedIn>
          </div>
        </Router>
      </ThemeProvider>
    </ClerkProvider>
  );
}

```

```
    } />
    <Route path="/onboarding" element={<Onboarding />} />
    <Route path="/preferences" element={<Preferences />} />
    <Route path="/budget" element={<Budget />} />
    <Route path="/analytics" element={
      <RequireOnboarding>
        <Analytics />
      </RequireOnboarding>
    } />
  </Routes>
</Layout>
</SignIn>
</div>
</Router>
</ThemeProvider>
</ClerkProvider>
);
}

export default App;
```

src/utils/api.js


```
import axios from 'axios';

const API_BASE_URL = process.env.REACT_APP_API_URL || 'http://localhost:5000/api';

const api = axios.create({
  baseURL: API_BASE_URL,
  headers: {
    'Content-Type': 'application/json',
  },
});

// Add auth token to requests
api.interceptors.request.use(async (config) => {
  const token = await window.Clerk?.session?.getToken();
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

export const groceryAPI = {
  getItems: (filters = {}) => api.get('/grocery', { params: filters }),
  createItem: (item) => api.post('/grocery', item),
  updateItem: (id, updates) => api.patch(`/grocery/${id}`, updates),
  deleteItem: (id) => api.delete(`/grocery/${id}`),
};

export const preferencesAPI = {
  get: () => api.get('/preferences'),
  save: (preferences) => api.post('/preferences', preferences),
};

export const budgetAPI = {
  get: () => api.get('/budget'),
  save: (budget) => api.post('/budget', budget),
};

export const userAPI = {
  getMe: () => api.get('/users/me'),
  completeOnboarding: () => api.patch('/users/onboarding'),
};

export const aiAPI = {
  generateList: (data) => api.post('/ai/generate-list', data),
};
```

```
export default api;
```

Continue with the remaining frontend components in the next section...

Next Steps

This covers the foundation and backend setup. Would you like me to continue with:

1. Frontend components (Dashboard, Onboarding, etc.)
2. Deployment configuration
3. Testing setup
4. Specific component implementation

Let me know which part you'd like to focus on next!